2019 16th Conference on Computer and Robot Vision (CRV)

Resource-Aware Optimization of DNNs for Embedded Applications

Alexander Frickenstein
Autonomous Driving
BMW Group
alexander.frickenstein@bmw.de

Christian Unger
Autonomous Driving
BMW Group
christian.unger@bmw.de

Walter Stechele
Electrical and Computer Engineering
Technical University Munich
walter.stechele@tum.de

***Abstract*—Despite their outstanding success in solving complex computer vision problems, Deep Neural Networks (DNNs) still require high-performance hardware for real-time inference. Therefore they are not applicable to low-cost embedded hardware, where memory resources, computational performance and power consumption are restricted. Furthermore, current approaches of fitting neural networks to embedded hardware are time consuming, inducing slow development cycles. To address these drawbacks and satisfy the demands of embedded hardware, this paper proposes a computationally efficient magnitude-based pruning scheme, based on a half-interval search, combined with effective weight sharing, fixed-point quantization, and lossless compression. The proposed solution can be utilized to generate an optimized model, either with respect to memory demand or execution time. For instance, the memory demand of LeNet is compressed about 385×. VGG16 is pruned by about 14.5×, whilst its computational costs are reduced by about 1.6× for a CPU-based application and 4.8× for an FPGA one.**

***Keywords*-Pruning; Quantization; Weight-Sharing; Coding; Embedded HW**

I. Introduction

Since the breakthrough of DNNs for image classification in the beginning of this decade (e.g. AlexNet [1], VGG [2] and , GoogleNet [3]), neural networks help to achieve outstanding performance in complex tasks, such as object detection [4], semantic segmentation [5] and intention detection [6], having in common their underlying DNNs for feature extraction and classification. Though, even the most efficient object detection algorithms (e.g. YOLO9000 [7] and SSD300[4]) require an NVIDIA GTX Titan X, which is not only costly (∼ € 1,500), but also has a high power consumption (250W peak). The commonly underlying VGG16 requires more than 527MB of memory storage and takes more than 15.5B multiply-accumulate (MAC) operations for the classification of one image. During the deployment process of computer vision based systems for autonomous driving (e.g. free-space detection) multiple criteria have to be taken in to consideration, likewise the sensor setup. Having a backbone-model which easily learns new complex features is beneficial in that phase. Hence, finding a way to reduce the memory demand and computational costs of DNNs is mandatory.

This paper extends the state-of-the-art *Deep Compression* proposed by Han et al. [8], improving efficiency of the training stage by a sophisticated pruning scheme, an optimized gradient descent implementation, and fine-grained weight sharing. Additionally, the performance on embedded hardware improves further using low-precision quantization of the resulting weights and activations.

After an initial training of the neural network, its weights are pruned in a layer-wise and iterative fashion, performing fine-tuning after each pruning step. After pruning, unpruned (non-zero) weights are clustered and represented by their prototype (centroid), which are then fine-tuned, using gradient descent, based on the original training objective. Finally, a fixed-point quantization of the prototypes (inspired by [9]) improves the computational and memory efficiency, while a lossless compression of the centroid labels in the weight matrices enables the efficient storing of the sparse weights and minimizes the network's memory footprint. The key contributions of this paper are in particular:

- Applying a binary search and an improved magnitude-based pruning method, to accelerate the greedy search for an optimal pruning rate.
- Varying the pruning order to generate an optimized model either with respect to memory demand or execution time.
- Performing optimized Stochastic Gradient Descent (SGD) with momentum reconstruction, leading to higher sparsity and higher accuracies during pruning.
- Using improved weight sharing with automatic layer-wise selection of the number of centroids, achieving better data locality on the embedded device.
- Utilizing fixed-point quantization of shared weights and activations reduces memory bandwidth and improves computational performance.

II. Related Work

The deployment of DNNs on embedded devices goes along with the optimization of the models in terms of the memory demand, the computational costs and the power consumption. A review of related optimization methods for DNNs is given below.

Pruning: LeCun et al. [10] proposed pruning as an adequate technique for compressing DNNs. Pruning describes the elimination of redundant and unused weights and neurons from a DNN, making the model more resource efficient while maintaining the prediction accuracy. Lately, Dong et al. [11] applied the second order derivatives of a layer-wise

978-1-7281-1838-3/19/$31.00 ©2019 IEEE
DOI 10.1109/CRV.2019.00011

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 29,2026 at 08:35:30 UTC from IEEE Xplore. Restrictions apply.

error function, utilizing high-performance parallel hardware (48 Intel Xeon CPUs) to prune recent DNNs. Furthermore, non-essential elements of the networks' calculations may be eliminated by adding a penalty term to the objective function [12], eventually causing the optimization to get stuck at a local minimum. Louizos et al. [13] applied group-wise sparsity, using an elaborate Bayesian approach, to utilize non-diagonal priors (estimated by the relaxation of the spike-and-slab prior). The reparametrization leads to a low variance of the gradients. According to Han et al. [14], it is also possible to discard weights and connections referring to a threshold, by extensively increasing the threshold causing the synapse density to decline. Moreover, Yu et al. [15] applied structured sparsity (e.g. removing entire rows of a kernel) for CPU-based implementations, using vectorized operations, e.g. single instruction multiple data (SIMD). As modern CPUs' SIMD registers are expanding over times (128-bit→256-bit→512-bit) larger pruning vectors are the result, making SIMD-based pruning less effective. The effect of structured and unstructured sparsity is studied in [16], whereas the application on dedicated hardware, e.g. FPGAs, is exploited in [17] and [18]. However, most of the proposed pruning methods are very resource or time-consuming. In this paper, Han et al.'s [8] *Deep Compression* scheme is improved and accelerated by using a binary search, being more efficient for large DNNs than the recently presented *Dynamic Network Surgery* [19], recovering important weights of the model dynamically.

Weight Sharing: The memory system's energy consumption is a critical aspect of an efficient embedded application [20]. Weight sharing is a proper technique to perform near-memory processing by placing data to memory storage which is close to the computation units. Chen et al. [21] applied a low-cost hash function to the training scheme to accomplish weight sharing. Differently, Han et al. [8] applied k-means clustering, thus weights are clustered and represented by their prototype. They constrain clustering to an uniform use throughout the model. In this work, the optimal number of centroids is determined layer-wise and assigned automatically.

Quantization: Recently announced high-performance hardware for deep learning is inherently using low-precision computations to improve the system's power-density, e.g. NVIDIA Turing, Google TPU and Intel Nervana NNP. The improvements of quantization are diverse: 1) reducing the execution time of applications, 2) compressing the model's weights, 3) keeping the footprint of special hardware-accelerators small and improving the energy efficiency. Linear quantization of weights and activations is well explored. Vanhoucke et al. [22] approximate weights and activations to use efficient fixed-point SSE[1]-instructions for CPUs. The model-output's signal-to-quantization-noise ratio is taken into consideration by Darryl et al. [23] to find the optimal bit width for the computations. Both approaches are limited to standard data types, i.e. int8. On the contrary, Gupta et al. [9] studied limited numerical precision for the inference and training of DNNs. Their scheme is chosen to be part of the proposed optimization methodology and is extended for the use with sparse and clustered weights. In contrast, Zhou et al. [24] enabled highly efficient bit shift operations for the use on dedicated hardware (i.e. FPGA), introducing a low-precision representation where weights are either powers of two or zero.

Compression based on Reinforcement Learning: Reinforcement Learning (RL) based compression schemes try to overcome the memory and computational complexity by defining reward functions for the optimization process. An agent is used to explore the design space whilst compressing a CNN model. Tien-Ju Yang et al. [25] propose an agent based scheme, called NetAdapt, which prunes convolutional filter of a CNN with respect to metrics for the hardware resource consumption. The more recent RL based approach AutoML, explored by Hihui He et al. [26], includes well studied heuristics (e.g. not pruning the first layer). Moreover, the RL based agent of AutoML prunes the given model layerwise with respect to a reward function taking the accuracy and the MACs into consideration. Differently speaking, more advanced reward functions are required for the optimization of DNNs. Especially, when compression is only one aspect of the deployment process. Also, multiple optimization techniques should be combined to maximize the efficiency of embedded applications. In brief, numerous optimization methods for an efficient implementation of DNNs are published. However, the optimization of large-scale DNNs is a time-consuming and resource intensive aspect of the deployment process. For example, Han et al. [14] spend 173h to prune AlexNet, whilst its initial training took 75h. For the greedy exploration of SqueezeNet [27], a 18,000 GPU super-computer is necessary [28]. For this reason, a fast and efficient optimization methodology is required in the rapidly changing domain of agile software development.

III. Resource-Aware Optimization

To deploy models on low-cost embedded hardware with limited resources, several aspects have to be examined. First, DNNs have to be highly compressed to fit the resource constraints of the hardware. Second, good data locality is essential for low-power applications [20]. Third, the deployment time of DNNs for embedded applications is an important aspect of agile software development. The proposed optimization methodology, visualized in Figure 1, is capable of compressing large-scale DNNs in a highly productive manner, whilst achieving state-of-the-art compression results. The running example shows the proposed four-stage optimization pipeline consisting of pruning, weight sharing, quantization and coding. It aims to reduce the memory demand and improves the computational costs of latency critical systems. The steps of the pipeline are described in detail below.

[1]Streaming SIMD Extensions (SSE) is an extension of the SIMD-instructions for x86 architectures

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 29,2026 at 08:35:30 UTC from IEEE Xplore. Restrictions apply.

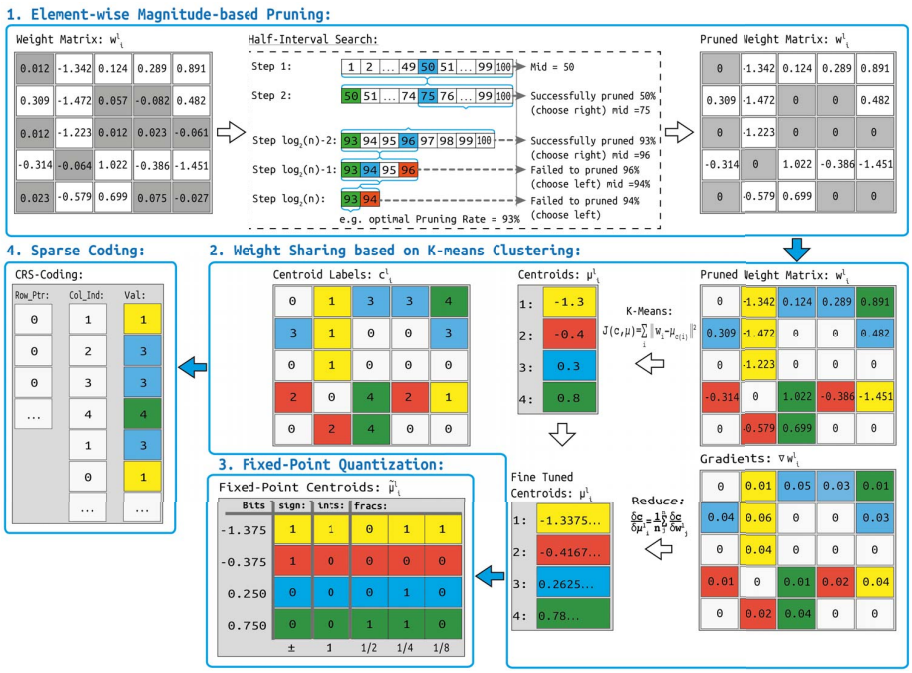


Fig. 1: Resource-aware optimization applied to a 5×5 convolutional kernel. 1) Finding the maximal pruning rate with half-interval search. 2) Clustering the sparse weights results in centroids μ_i^l and corresponding labels c_i^l. Centroid fine tuning improves the classification capabilities of the DNN. 3) Fixed-point quantization levels the precision of weights and activations to fit the approximation of the shared weights. 4) Storing the centroids-labels c_i^l using Compressed Row Storage (CRS) coding.

A. *Element-wise Magnitude-based Pruning*

Half-Interval Search: This section introduces a novel magnitude-based pruning scheme, removing unimportant and redundant weights, which are smaller than a threshold, from the model. By using a layer-wise half-interval (binary) search, the optimal pruning rate is explored more quickly compared to other approaches (e.g. [14] and [19]). The proposed pruning scheme generates an optimized model, either with respect to memory demand or execution time. Let pr_m denote the actual pruning rate. $pr_{m_i} = (pr_{s_i} + pr_{e_i})/2$ is iterated with $i \in [0, \log_2(n) - 1]$ and $pr_{s_0} = 0$, $pr_{e_0} = 100\%$. According to Figure 1, the pruning rate is increased with each iteration ($pr_{s_{i+1}} = pr_m$) if the classification accuracy is maintained or within a noise reduction bound. Moreover, the retrained weights are used for the next iteration $(i+1)$. If the evaluation fails, the current weights are discarded and the parameters of the last successful iteration are restored and it follows $pr_{e_{i+1}} = pr_m$ for the next iteration. Conventional iterative pruning [14] of a DNN, conducted by slowly increasing a threshold, can be expressed as a linear search for the optimal pruning rate with the time complexity $\mathcal{O}_i = n \cdot l \cdot e$, with the number of layers l, the number of epochs e (required to retrain the model after each pruning step) and the number of iterations n. Han et al. [14] apply an analysis of the pruning sensitivity to set a lower bound for n to accelerate the extensive search. In other words, the initial pruning rate is set, which maintains the accuracy of the model. Figure 2 illustrates the robustness of VGG16 due to pruning, whereas the first fully-connected layer Fc6 (blue) conserves the accuracy and Conv1_1 (purple) is sensitive to pruning. In contrast, the time complexity $\mathcal{O}_b = (\log_2(n)+1) \cdot l \cdot e$ of the binary search based pruning scheme is reduced drastically compared to the linear search, which holds especially for DNNs with several layers, e.g. VGG16. The binary search cuts down the total number of required iterations from n to $log_2(n)$, resulting in a higher rate of convergence for model compression.

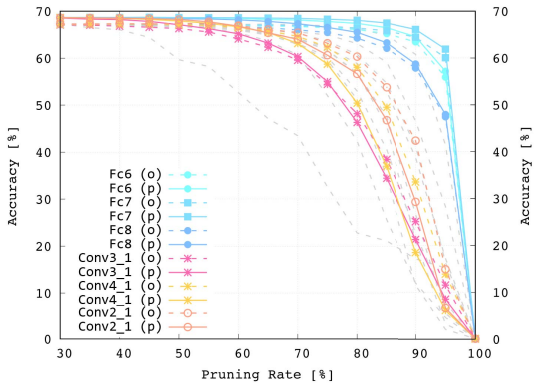


Fig. 2: Pruning sensitivity of VGG16. Sensitivity changes during the process of pruning. (o) denotes the sensitivity without prior pruning and (p) after pruning.

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 29,2026 at 08:35:30 UTC from IEEE Xplore. Restrictions apply.

Pruning Order: Pruning is applied to the convolutional (Conv) and fully-connected (Fc) layers. Conv-layers usually have a low number of weights compared to the number of MAC-operations [29]. In contrast, Fc-layers are more memory intensive, as every input neuron is connected to the individual output neurons via synapses. The MAC-operation, defined as $a \leftarrow a + (b \cdot c)$, is the most essential operation, applied to perform the convolutions in Conv- and Fc-layers. Varying the pruning order can be utilized to generate an optimized model either with respect to memory demand or execution time. The distinction in the process of removing the majority of the weights is that memory intensive layers are pruned at first, or analogously, computationally heavy layers are pruned at first to enhance the performance of a model.

In order to achieve optimal pruning results for a memory or performance based optimization, the pruning sensitivity analysis of Han et al. [14] should not be used, as the sensitivity of the layers are changing their order during pruning. As the initial pruning order is obsolete after starting the pruning process. Figure 2 illustrates, that during pruning, some layers are getting more sensitive (Conv4_3 (yellow)), whereas others are getting more robust (Conv3_1 (pink)). This would result in the necessity to analyze the models' sensitivity after each layer – which is computationally too expensive. Thus, the pruning order is fixed set initially based on the desired optimization goal. Said differently, start pruning with computationally heavy layers for a low latency application (autonomous driving) or start pruning memory-intensive layers for a storage critical application (smartphone apps). In Table II and III, the pruning order for the memory and performance-based pruning order is given in brackets for LeNet and VGG16.

Retraining by SGD with Momentum: As part of the magnitude-based pruning method, it is crucial to retrain the remaining weights and strengthen their connections after a pruning step [14]. This holds especially true when the pruning steps are large and using binary search. SGD with momentum [30] is chosen to retrain the non-zero weights and connections of the pruned DNNs. Eq. 1 and 2 express the well-known update-rule with the objective function $f(\Theta)$, the learning rate ε, the momentum μ, the accumulator v_t and the constraint for sparsity. In detail, the update v_{t+1} is also set to zero if $\Theta = 0$. Thus, the gradient $\nabla f(\Theta_t)$ is not applied to the pruned weights. Applying L2 regularization to the loss is proven to be a proper regularization technique for pruning [14].

$$v_{t+1} = \begin{cases} \mu v_t - \varepsilon \nabla f(\Theta_t) & \text{if } \Theta_t \neq 0 \\ 0 & \text{else} \end{cases} \tag{1}$$

$$\Theta_{t+1} = \Theta_t + v_{t+1} \tag{2}$$

The selection of SGD with momentum is based on several aspects. Firstly, SGD with momentum improves the speed of convergence efficiently compared to vanilla SGD [31] – which is well-known. Moreover, the optimizer is less computationally intensive compared to second order optimizers, like the quasi-Newton-based optimization L-BFGS using an approximation of the inverse Hessian [32]. Lastly, the momentum consist of information about strong connections in the CNN, which are essential for the alternating pruning and retraining phases of the proposed method.

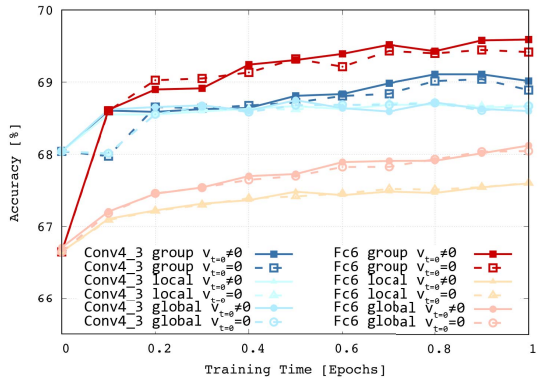
Momentum Reconstruction: When compressing a model, it happens that the update v_t is missing for the model. Therefore, only weights and biases can be utilized. The reasons are many fold: 1) a different optimizer (e.g. Adam [33]) is used for initial training, 2) a reference model from the Caffe Model Zoo[2] is used or 3) a different layer type was pruned before (Conv→Fc). Thus, the reconstruction of the update v_t, see Eq. 1, is essential for a crisp pruning. Sutskever et al. [34] have demonstrated that a proper weight initialization and available momentum have significant importance for a sufficient reduction of the entropy – which is well known. The proposed work shows that the momentum is also essential for magnitude based pruning. Meanwhile, Smith et al. [35] discovered important aspects required for a good generalization of SGD with momentum, using the Bayesian principle. They have shown that noise in training drives the optimization away from local minima, whilst the optimal batchsize also improves the accuracy of the test dataset. In this context, they proposed an approximation of the noise scale g for SGD with momentum, where N denotes the size of the training dataset and the batchsize B, see Eq. 3. This causes to improve the pruning rate of the binary search-based scheme.

$$g \approx \frac{\varepsilon N}{B(1-\mu)} \xrightarrow{\mu=0} g \approx \varepsilon \left(\frac{N}{B} - 1 \right) \tag{3}$$

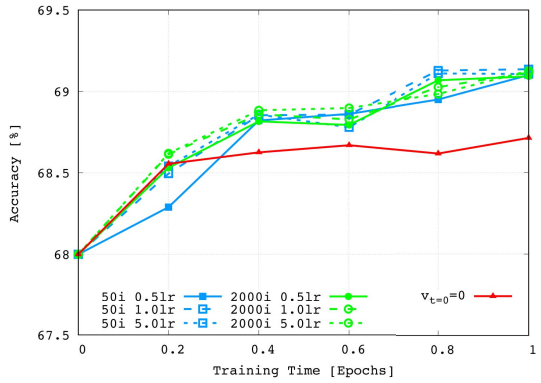
If the update v_t is not reconstructed prior to pruning, the noise scale is reduced as expressed in Eq. 3 and a twofold problem occurs. Firstly, the learning noise is drastically reduced leading the optimization to a local minimum. Secondly, the absence of the update accumulators v_t^l in the first iterations (in combination with pruning) weakens the connectivity of relevant synapses, resulting in poor pruning rates and low accuracy. In Figure 3a, pruning is applied to a Conv-layer (blue) and an Ip-layer (red), at which the layers are retrained once with or without ($v_{t=0} = 0$) the reconstructed update v_t. Group-wise retraining, e.g. all Conv-layers are retrained if a Conv-layer is pruned and vice versa for the Ip-layers, results in the best pruning rates compared to local and global retraining, see Figure 3a. It is alluring to retrain all layers at once, including the currently pruned one, as the accuracy of the model and the pruning results are promising at the beginning. Regrettably, the remaining unpruned layers compensate the missing neurons and can only be sparsely pruned afterwards. Moreover, the remaining neurons suffer the problem of vanishing gradients [36]. However, experiments showed that a well-working heuristic applies a conservative learning rate to recover the models accuracy in the retraining phase, helping the bound of the entropy (loss) to be kept aligned, see Figure 3b. Moreover, when optimizing large-scale DNNs (e.g. VGG16) with the proposed methodology, the original batchsize might be too large for the GPU's memory.

[2] http://caffe.berkeleyvision.org/model_zoo.html

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 29,2026 at 08:35:30 UTC from IEEE Xplore. Restrictions apply.



(a) Training of Conv4_3 and Ip6 w a. w/o update v_t.



(b) Group-wise retraining of Conv4_3 (50% pruned).

Fig. 3: Heuristics for pruning and retraining of DNNs (VGG16), with i: Training iterations, lr: Base learning rate.

By reducing the batchsize, the learning noise increases. Thus, the learning rate ε and the momentum μ have to be adapted pursuant to Eq. 3. Figure 4 illustrates a hyper-parameter search for LeNet, where the overall-pruning rate is evaluated with respect to the time required for optimization and the allowed accuracy degradation. Pruning has a sharp transition between the compression results and the model accuracy. Moreover, as weight-reconstruction (see Figure 3a) is applied the required time for retraining vanishes and accelerates compression. Briefly, optimization is a trade-off between compression-rate, accuracy and optimization time.

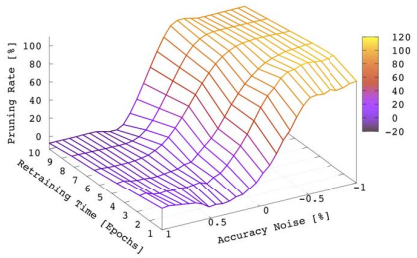


Fig. 4: Relation between training time, accuracy and pruning rate for LeNet.

B. Weight Sharing based on K-means Clustering

K-means clustering [37] (weight-sharing) is a kind of approximation and used to decrease the traffic between the memory-device and the computing-engine.
K-means iteratively minimizes the Euclidean distance of the prototypes $\mu_{c(i)}$ and the weights w_i^l, leading to an approximation of the weights $\tilde{w}_i^l = \mu_{c(i)} \approx w_i^l$.

In Figure 1, an exemplary application of clustering is given, where the memory footprint of the weights w^l is reduced as the convolutional kernels consist only of the centroids labels c_i^l and a look-up table for the centroids μ_i^l (prototypes). Han et al. [8] have utilized k-means clustering for compression. Different to their scheme, where Conv-layers are assigned uniformly to 256 centroids and Ip-layers are assigned to 32 centroids, the optimal number of centroids is evaluated layer-wise for the presented method. Thus, the model's accuracy is better maintained and higher compression results are achieved. In the experiments, a $\sim 20\%$ improved compression rate is achieved compared to [8]. As a matter of fact, the experiments also demonstrate that group-wise (e.g. centroids for all Conv-layers) or uniform clustering of the entire DNN performs poorly.

Centroid Fine Tuning: By decreasing the number of centroids, the accuracy of the network would be diminished. Therefore, centroid fine tuning [8] is used to increase the accuracy of the network. Though, the implementation was changed in the fundamental point of not only accumulating the gradients of the individual centroids. Instead, the arithmetic mean of the gradients is computed and applied to the prototypes, illustrated in Figure 1 by gradients of the same color. This modification is necessary to achieve better results in accuracy. Centroid fine tuning is applied in three steps: 1) Weight gradients $\partial c / \partial W^l$ are computed with respect to the cost-function c (cross entropy loss), 2) gradients $\partial c / \partial W_j^l$ are grouped by their corresponding centroids μ_i and 3) the centroid gradients $\partial c / \partial \mu_i^l$ are computed referred to Eq. 4. Throughout the fine-tuning weights do not change their assignment to a specific centroid. In contrast, applied by [8], a summation based update depends on the layer's size. Drawing conclusions from the experimental results, it is apparent that large layers are occasionally trained strong than smaller ones.

$$\frac{\partial c}{\partial \mu_i^l} = \frac{1}{n} \sum_j^n \frac{\partial c}{\partial W_j^l} \tag{4}$$

C. Fixed-Point Quantization

Quantization leads to a hardware friendly implementation of DNNs on embedded systems, reducing the footprint of hardware components, lowering the memory bandwidth and improving the performance, where vectorization on CPUs by SIMD-instructions is proper option. Due to weight sharing, the centroids μ_i^l are already approximations of the original weights w_i^l, making them a proper candidate for quantization. In this optimization-step the shared weights μ_i^l are brought into a fixed-point format $\tilde{\mu}_i^l$ with the notation $< \mathtt{S}, \mathtt{IB}, \mathtt{FB} >$, see Eq. 5 [9], with a sign bit $\mathtt{S}$, the number of integer bits

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 29,2026 at 08:35:30 UTC from IEEE Xplore. Restrictions apply.

IB and fractional bits FB. The precision $\epsilon = 2^{-\mathrm{FB}}$ is limited by FB and the upper and lower bounds of $\tilde{\mu}_i^l$ are $\pm 2^{\mathrm{IB}} - 2^{\mathrm{FB}}$. Values beyond the bounds are clipped. Round-to-nearest, see Eq. 5 (line 3 and 4), is used to approximate the weights of the models. Differently, Gupta et al. [9] use stochastic rounding as they train the models after quantization, whereas round-to-nearest would set values between $[-\epsilon/2, \epsilon/2]$ to zero. $\lfloor \mu_i^l \rfloor$ denotes the largest integer multiplier of ϵ. In brief, quantized weights are not retrained after quantization, as the approximation is primarily limited by weight sharing.

$$\tilde{\mu}_i^l = \begin{cases} -2^{\mathrm{IB}} - 2^{\mathrm{FB}} & \text{if } \mu_i^l < -2^{\mathrm{IB}} - 2^{\mathrm{FB}} \\ 2^{\mathrm{IB}} - 2^{\mathrm{FB}} & \text{if } \mu_i^l > 2^{\mathrm{IB}} - 2^{\mathrm{FB}} \\ \lfloor \mu_i^l \rfloor & \text{if } \lfloor \mu_i^l \rfloor \leq \mu_i^l \leq \lfloor \mu_i^l \rfloor + \frac{\epsilon}{2} \\ \lfloor \mu_i^l \rfloor + \epsilon & \text{if } \lfloor \mu_i^l \rfloor + \frac{\epsilon}{2} < \mu_i^l \leq \lfloor \mu_i^l \rfloor + \epsilon \end{cases} \quad (5)$$

D. Compressed Sparse Weights

Finally, a lossless coding method is applied to the pruned, clustered and quantized centroid labels c_i^l. Coding is not applied to the quantized centroids $\tilde{\mu}_i^l$ as it is obligatory to have an efficient near-memory access to the values. The proposed scheme allows a combination of Huffman [38] and CRS coding, also known as Harwell-Boeing sparse matrix format [39]. Experiments have shown that CRS coding results in higher compression rates, compared to Huffman-coding, the more parameters are zero (pruned). In contrast, the compression results of Huffman-coding interfere with the number of clusters c, the parameter distribution and the pruning rate p. An approximation of the memory demand K_H is given in Eq. 6, where n denotes the number of weights. The value of absolute term $k = 0.95$ is determined from the experiments. Figure 1 shows the three vectors Val, Col_{Ind} and Row_{Ptr}, specifying a sparse matrix of the centroid labels c_i^l and the corresponding centroid labels c_i^l. However, the vector Val stores all centroid labels c_i^l, whereas Col_{Ind} stores its column indices. Finally, the vector Row_{Ptr} contains the non-zero elements in each row. As a matter of fact, the more coding techniques are applied to the weights, the more decoding is required before inference.

$$K_H \approx n \cdot (1 + k \cdot log_2(c) \cdot (1 - p))[bits] \quad (6)$$

IV. Experimental Results

The DNNs are trained and optimized with a standard PC with an Intel i7-7820C CPU, 32GB DDR4 RAM and an NVIDIA GTX 1080 Ti GPU, using Tensorflow v1.8 and Cuda8. Low latency applications are applied on a ARM Cortex A-53 (Raspberry Pi) and an Altera Arria 10 FPGA. In Table I, II and III, the results for the optimization of the memory demand or the execution time are colored red or blue.

A. Memory Savings

LeNet trained on MNIST: As a first experiment, the historical architecture of LeNet[40], trained and evaluated on MNIST [40], is used as an elementary baseline for the investigation of more or less binary data. Pruning is carried out, either to reduce the number of MAC-operations or the memory demand of the DNN. A detailed overview of the compression parameters is given in Table II. The summary implies, that the performance based pruning order (see third column in brackets) results in less MAC-operations, compared to the memory based pruning order (see second column in brackets), whilst the memory savings are slightly diminished. The proposed four-stage optimization methodology leads to a 4.35kB (385× compression) version of LeNet. In Table II and III, *Non-Zero* denotes the number of unpruned weights [%] and *MACs* denotes the number of MAC-operations.

VGG16 trained on ImageNet: VGG16 is commonly used in state-of-the-art object detectors, resulting in the demand for a good compression rate of the model. VGG16 is trained and evaluated on ILSVRC12 [1]. Furthermore, as the compressed version of VGG16 might be useful for embedded enthusiasts, the optimized models are openly available. A summary of the performance and the memory based optimization for VGG16 is given in Table III.

B. Low Latency Application

In the previous subsection the memory improvements by using the optimization pipeline are analysed and compared to recent approaches of the community. Hereinafter, the performance of an implementation in resource aware embedded hardware is compared, showing the capabilities of the optimization pipeline. The performance-based optimization leads in low latency applications on different type accelerators.

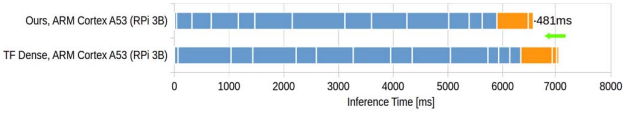


Fig. 5: Performance improvement of ARM Cortex A-53 based implementation of VGG16. (TF-Dense incl. Arm Neon instr.)

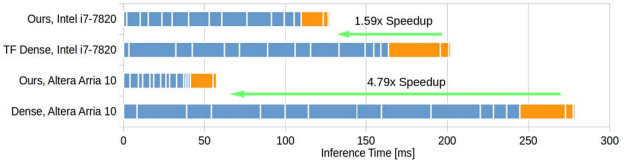


Fig. 6: Performance improvement of CPU and FPGA-based implementation of VGG16. (Conv-layers: blue, Fc-layer: orange).

For the evaluation of the latency, the batchsize of the model is set to 1.

Intel i7-7820: In Figure 6 the layer-wise execution time of the reference model is compared to our performance optimized implementation of VGG16, see third column of Table III. Compared to the baseline model and using highly efficient Tensorflow implementation the performance optimized model is 1,59× faster. Because of the large cache memory (L1d cache: 32K, L2 cache: 1024K, L3 cache: 11264K) the CPU is capable of performing well on the sparse data and memory stalls does not occur. Vectorization by using 256-bit SIMD instructions are used to run the spares layers of VGG16.

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 29,2026 at 08:35:30 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Compression results for LeNet and VGG16: The accuracy of compressed models is given along with the baseline accuracy. The four-stage optimization outperforms existing publications. Numbers marked with $^+$ are best-case estimations.

Model	Top-1 Acc.	Δ Acc.	Params.	Comp. Rate	MACs	Retrain [Epochs]	([Time])
LeNet reference	98.9%*	+0.0%	1,720kB		2.3M	∼ 100	(21min)
DC [8]	99.2%	+0.3%	44kB	39×	415k	∼ 160	(34min)
DNS [19]	99.1%	+0.2%	16kB	108×	91k	- - -	- - -
BC [13]	99.0%	+0.1%	10.5kB	361×	106k$^+$	- - -	- - -
L-OBS [11]	98.7%	-0.2%	17kB	101×	78k	∼ 8	(4min)
LeNet Mem. (Ours)	98.9%*	+0.0%	**4.4kB**	**385×**	210k	∼ 112	(24min)
LeNet Perf. (Ours)	98.9%*	+0.0%	10.0kB	172×	**57k**	∼ 112	(24min)
VGG16 reference	68.5%	+0.0%	138.4M		15.47B	∼ 90	(7.5d)
LWC [14]**	68.7%	+0.2%	10.4M	13.1×	5.04B	∼ 4,950	(412d)
L-OBS [11]**	62.7%	-5.8%	10.4M	13.1×	5.45B	∼ 19	(3.2d)
DNS [19]**	36.6%$^+$	-31.9%	10.4M	13.1×	- - -	∼ 230	(19.2d)
VGG16 Mem. (Ours)**	68.4%	-0.1%	**9.5M**	**14.5×**	7.14B	∼ 192	(16d)
VGG16 Perf. (Ours)**	68.5%	+0.0%	19.4M	7.1×	**3.54B**	∼ 192	(16d)

* 5,000 images of the training set are used for validation. Otherwise, the reported accuracy is 99.1%.
** Optimization of VGG16 is limited to pruning.

TABLE II: Compression parameters for LeNet: Column 1) describes the baseline, 2) a performance-based optimization and 3) a memory-based compression. Pruning orders are given in brackets.

Layer	Params.	MACs	Perf. Pruning Non-Zero	Perf. Pruning MACs (P)	Memory Pruning Non-Zero	Memory Pruning MACs (P)	c_i^l bits	$\tilde{\mu}_i^l$ bits S; IB; FB	Compr. Rate
Conv1	0.5k (4)	288k (3)	9.8%	28.2k	62.0%	178.6k	7	1;0;9	2.5×
Conv2	25k (2)	1.6M (1)	1.3%	20.2k	1.8%	28.8k	5	1;0;9	131.7×
Fc1	400k (1)	400k (2)	1.5%	6.0k	0.5%	2.0k	4	1;0;9	704.7×
Fc2	5k (3)	5k (4)	49.0%	2.5k	7.6%	0.4k	6	1;0;9	30.4×
Total:	430.5k 1,684kB	2.3M	**2.5%**	**57.5k** 2.5%	**0.7%**	**209.8k** 9.1%			**385×** **4.35kB**

TABLE III: Pruning of VGG16: Column 1) describes baseline VGG16, 2) benchmark, 3) a performance-based optimization and 4) a memory-based compression. Pruning orders are given in brackets.

Layer	Params.	MACs	LWC[14] Non-Zero	LWC[14] MACs (P)	Perf. Pruning Non-Zero	Perf. Pruning MACs (P)	Memory Pruning Non-Zero	Memory Pruning MACs (P)
Conv1_1	1.8k (16)	87M (14)	58%	50.3M	67%	58.0M	70%	60.6M
Conv1_2	37k (15)	1.85B (6)	22%	406.9M	20%	367.0M	57%	1.05B
Conv2_1	74k (14)	925M (9)	34%	314.5M	33%	305.2M	60%	554.9M
Conv2_2	148k (13)	1.85B (5)	36%	665.9M	27%	499.4M	59%	1.09B
Conv3_1	295k (12)	925M (8)	53%	490.2M	27%	249.7M	66%	610.4M
Conv3_2	590k (10)	1.85B (4)	24%	443.9M	18%	332.9M	36%	665.9M
Conv3_3	590k (11)	1.85B (3)	42%	776.9M	25%	462.4M	73%	1.35B
Conv4_1	1.2M (9)	925M (7)	32%	296.0M	28%	259.0M	40%	369.9M
Conv4_2	2.4M (5)	1.85B (2)	27%	499.4M	17%	314.5M	24%	443.9M
Conv4_3	2.4M (7)	1.85B (1)	34%	628.9M	16%	296.0M	30%	554.9M
Conv5_1	2.4M (8)	426M (12)	35%	161.8M	32%	148.0M	32%	148.0M
Conv5_2	2.4M (6)	426M (11)	29%	134.1M	26%	120.2M	19%	87.9M
Conv5_3	2.4M (4)	462M (10)	36%	166.5M	23%	106.4M	31%	143.4M
Fc6	103M (1)	103M (13)	4%	4.1M	8%	8.2M	3%	3.1M
Fc7	16.8M (2)	16.8M (15)	4%	671.1k	29%	6.2M	6%	1.0M
Fc8	4.1M (3)	4.1M (16)	23%	942.1k	70%	2.9M	19%	0.8M
Total:	138.4M 527.8MB	15.5B	**7.6%**	**5.04B** **32.6%**	14%	**3.54B** **22.9%**	**6.9%**	7.14B

ARM Cortex A-53: The next comparison is performed on a low-power embedded ARM Cortex A-53 (Raspberry Pi 3b). In Figure 5 the performance of the optimized CNN is compared to the Tensorflow-based implementation. The performing one inference cycle on the RPi is 481ms faster if the performance optimized model is used rather then the Tensorflow baseline model. As the RPi has a very small cache memory (L1d cache: 32k, L2 cache: 128k), memory stalls are more likely than on a larger CPU, e.g. Intel i7-7820.

Altera Arria 10: The best performance is achieved by using high level synthesis on an Altera Arria 10. The benefits of using FPAGs for acceleration are twofold: First, memory stalls are avoided by an efficient data fetching pipeline, which receives data over the PCI-bus and prepares the activations and parameters to the processing units. Thus, memory stalls are prevented. Second, the degree of parallelization suits the layers demand. Briefly, the proposed model is 4.8× more efficient compared to the reference dense implementation.

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 29,2026 at 08:35:30 UTC from IEEE Xplore. Restrictions apply.

V. Conclusion

An efficient and resource-aware optimization methodology is key to enable the utilization of DNNs on low-cost embedded systems, lacking of memory and computational resources. The four-stage optimization pipeline, consisting of a half-interval based pruning scheme with momentum reconstruction, weight-sharing, fixed-point quantization and lossless compression, allows the deployment of DNNs in a timely manner. Bringing the optimization methods together has shown superior compression results for LeNet with a 385× reduction of the memory demand or a 40.4× abatement of the computational costs. State-of-the-art compression rates are achieved for VGG16 in a swift time, using resource-aware hardware. In regard of the execution time of DNNs, the performance driven pruning order is advantageous over previous publications, to the best of our knowledge.

References

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *NIPS*, p. 2012. 1, 6

[2] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *CoRR*, 2015. 1

[3] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *CVPR*, June 2015, pp. 1–9. 1

[4] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. E. Reed, C. Fu, and A. C. Berg, "SSD: single shot multibox detector," *ECCV*, 2015. 1

[5] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *CVPR*, June 2015, pp. 3431–3440. 1

[6] Y. Li, W. Ouyang, X. Wang, and X. Tang, "Vip-cnn: Visual phrase guided convolutional neural network," in *CVPR*, July 2017. 1

[7] J. Redmon and A. Farhadi, "Yolo9000: Better, faster, stronger," *CVPR*, pp. 6517–6525, 2017. 1

[8] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural network with pruning, trained quantization and huffman coding," *CoRR*, vol. abs/1510.00149, 2016. 1, 2, 5, 7

[9] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, "Deep learning with limited numerical precision," in *ICML*, 2015, pp. 1737–1746. 1, 2, 5, 6

[10] Y. LeCun, J. S. Denker, and S. A. Solla, "Optimal brain damage," in *NIPS*, D. S. Touretzky, Ed., 1990, pp. 598–605. 1

[11] X. Dong, S. Chen, and S. J. Pan, "Learning to prune deep neural networks via layer-wise optimal brain surgeon," in *NIPS*, 2017. 1, 7

[12] M. D. Collins and P. Kohli, "Memory bounded deep convolutional networks," *CoRR*, vol. abs/1412.1442, 2014. 2

[13] C. Louizos, K. Ullrich, and W. Max, "Bayesian compression for deep learning." in *NIPS*, 2017, pp. 3290–3300. 2, 7

[14] S. Han, J. Pool, J. Tran, and W. J. Dally, "Learning both weights and connections for efficient neural networks," in *NIPS*, 2015. 2, 3, 4, 7

[15] J. Yu, A. Lukefahr, D. J. Palframan, G. S. Dasika, R. Das, and S. A. Mahlke, "Scalpel: Customizing dnn pruning to the underlying hardware parallelism," 2017, pp. 548–560. 2

[16] H. Mao, S. Han, J. Pool, W. Li, X. Liu, Y. Wang, and W. J. Dally, "Exploring the regularity of sparse structure in convolutional neural networks," *CoRR*, vol. abs/1705.08922, 2017. 2

[17] T. Posewsky and D. Ziener, "A flexible fpga-based inference architecture for pruned deep neural networks," in *ARCS*, 2018, pp. 311–323. 2

[18] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, and M. A. Horowitz, "EIE: efficient inference engine on compressed deep neural network," *ICSA*, 2016. 2

[19] Y. Guo, A. Yao, and Y. Chen, "Dynamic network surgery for efficient dnns," in *NIPS*, 2016, pp. 1387–1395. 2, 3, 7

[20] M. Horowitz, "1.1 computing's energy problem (and what we can do about it)," in *ISSCC*, Feb 2014, pp. 10–14. 2

[21] W. Chen, J. T. Wilson, S. Tyree, K. Q. Weinberger, and Y. Chen, "Compressing neural networks with hashing trick," in *ICML*, 2015. 2

[22] V. Vanhoucke, A. Senior, and M. Z. Mao, "Improving the speed of neural networks on cpus," in *NIPS*, 2011. 2

[23] D. D. Lin, S. S. Talathi, and V. S. Annapureddy, "Fixed point quantization of deep convolutional networks," in *ICML*, 2016. 2

[24] A. Zhou, A. Yao, Y. Guo, L. Xu, and Y. Chen, "Incremental network quantization: Towards lossless cnns with low-precision weights," *ICLR*, 2017. 2

[25] T.-J. Yang, A. G. Howard, B. Chen, X. Zhang, A. Go, M. B. Sandler, V. Sze, and H. Adam, "Netadapt: Platform-aware neural network adaptation for mobile applications," in *ECCV*, 2018. 2

[26] Y. He, J. Lin, Z. Liu, H. Wang, L.-J. Li, and S. Han, "Amc: Automl for model compression and acceleration on mobile devices," in *ECCV*, September 2018. 2

[27] F. N. Iandola, M. W. Moskewicz, K. Ashraf, S. Han, W. J. Dally, and K. Keutzer, "Squeezenet: Alexnet-level accuracy with 50x fewer parameters and ¡1mb model size," *CoRR*, vol. abs/1602.07360, 2017. 2

[28] F. N. Iandola and K. Keutzer, "Keynote: small neural nets are beautiful: enabling embedded systems with small deep-neural- network architectures," *CODES+ISSS*, pp. 1–10, 2017. 2

[29] A. Canziani, A. Paszke, and E. Culurciello, "An analysis of deep neural network models for practical applications," *CoRR*, vol. abs/1605.07678, 2016. 4

[30] B. Polyak, "Some methods of speeding up the convergence of iteration methods," *USSR Computational Mathematics and Mathematical Physics*, vol. 4, pp. 1–17, 12 1964. 4

[31] N. Qian, "On the momentum term in gradient descent learning algorithms," *Neural Networks*, vol. 12, pp. 145–151, Jan. 1999. 4

[32] Y. Xiao, Z. Wei, and Z. Wang, "A limited memory bfgs-type method for large-scale unconstrained optimization," *Computers & Mathematics with Applications*, vol. 56, pp. 1001–1009, 2008. 4

[33] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," vol. abs/1412.6980, 2015. 4

[34] I. Sutskever, J. Martens, G. Dahl, and G. Hinton, "On the importance of initialization and momentum in deep learning," in *ICML*, Jun 2013, pp. 1139–1147. 4

[35] S. L. Smith and Q. V. Le, "A bayesian perspective on generalization and stochastic gradient descent," *ICLR*, 2017. 4

[36] Y. Bengio, P. Y. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE transactions on neural networks*, vol. 5 2, pp. 157–66, 1994. 4

[37] H. Steinhaus, "Sur la Division des Corp Materiels en Parties," *Bull. Acad. Polon. Sci*, vol. 1, pp. 801–804, 1956. 5

[38] D. A. Huffman, "A method for the construction of minimum-redundancy codes," vol. 40, Sept 1952, pp. 1098–1101. 6

[39] I. S. Duff, R. G. Grimes, and J. G. Lewis, "Users' Guide for the Harwell-Boeing sparse Matrix collection (Release 1)," 1992. 6

[40] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, no. 11, pp. 2278–2324, Nov 1998. 6

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 29,2026 at 08:35:30 UTC from IEEE Xplore. Restrictions apply.